

Universitatea Tehnica din Cluj-Napoca
Departament Calculatoare

Programming Techniques

Software Engineering Process

T. Cioara, V. Chifu, C. Pop, D. Mitrea
2026

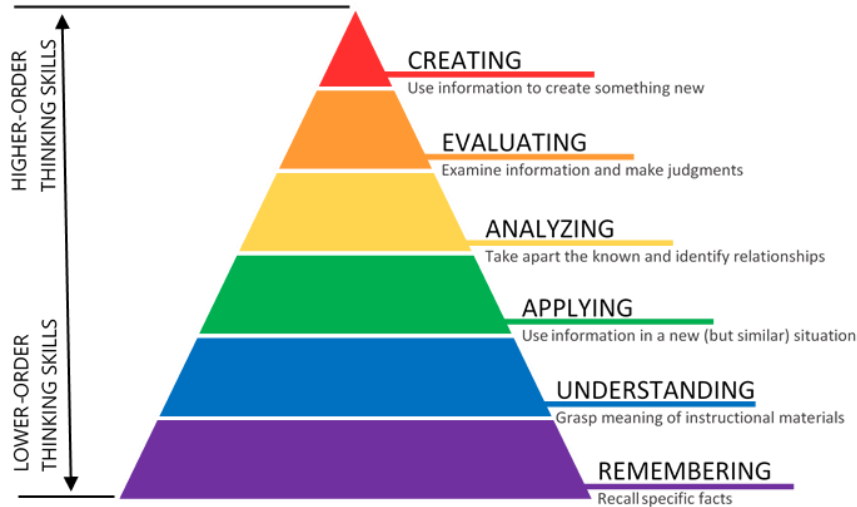
Problem Solving

- What do all have in common?



Learning for Problem Solving

BLOOM'S TAXONOMY – COGNITIVE DOMAIN (2001)



Superficial Learning (Lower Bloom Levels Dominance)

LLMs can:

- Instantly generate solutions (Apply level)
- Provide full system designs (Create level)

Students may skip:

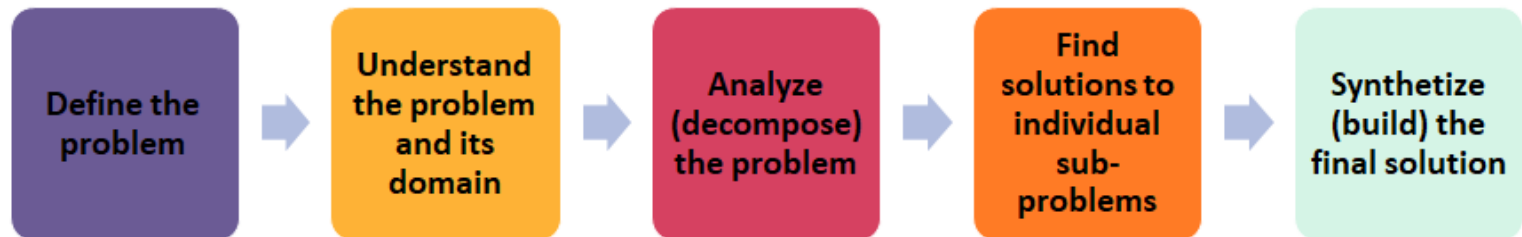
- Understanding, Analyzing, Evaluating

Problem: Illusion of competence without deep comprehension.

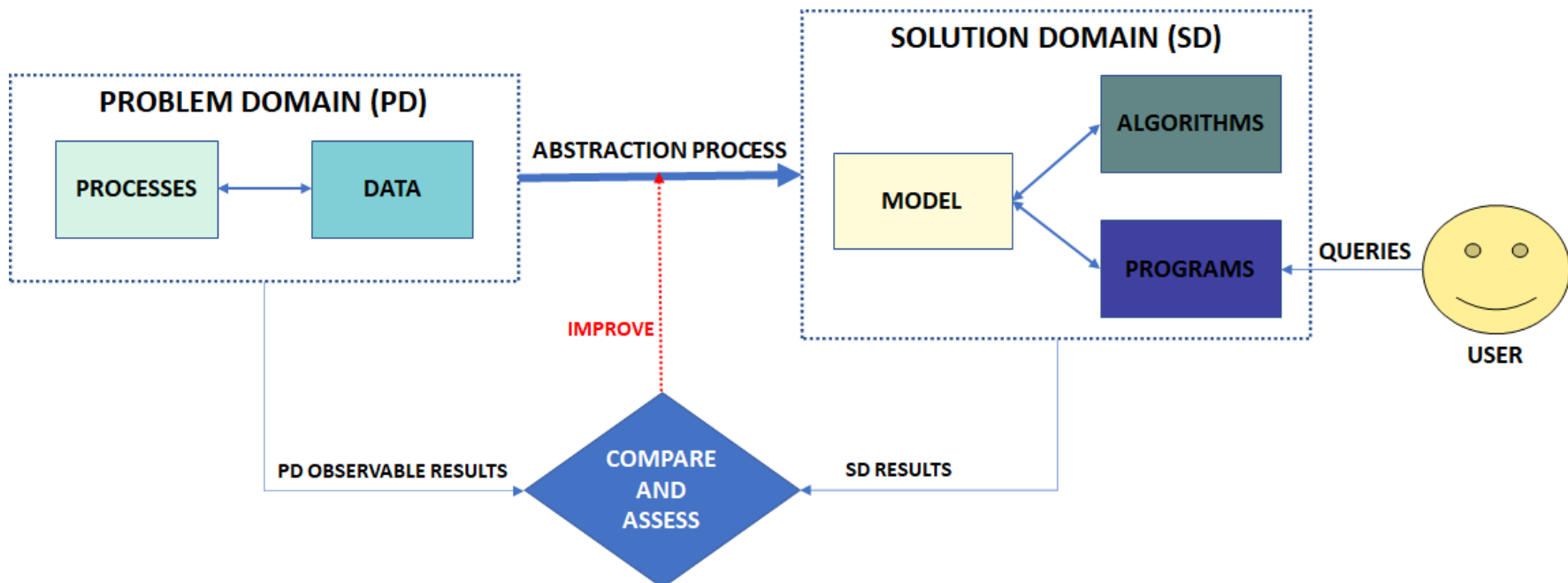
Bloom Level	Cognitive Skill	OOP Focus	Type of Problem Solving	Example Assessment Task
Remembering	Recall facts & definitions	OOP principles, syntax, terminology	Recognizing known concepts	Define encapsulation; list OOP pillars
Understanding	Explain ideas	Relationships between classes, inheritance, polymorphism	Explaining how a solution works	Explain method overriding with example
Applying	Use knowledge in structured tasks	Implementing classes, constructors, methods	Solving well-structured problems	Implement Lab Assignments
Analyzing	Break down systems	Identifying class relationships, debugging, refactoring	Decomposing complex problems	Analyze a system and identify design errors
Evaluating	Make justified decisions	Choosing inheritance vs composition, assessing design quality	Comparing alternative solutions	Justify best design for given scenario
Creating	Design new systems	Full OOP system development, applying SOLID principles	Solving open-ended problems	Design and implement a complete OOP application

Problem Solving

- Main steps

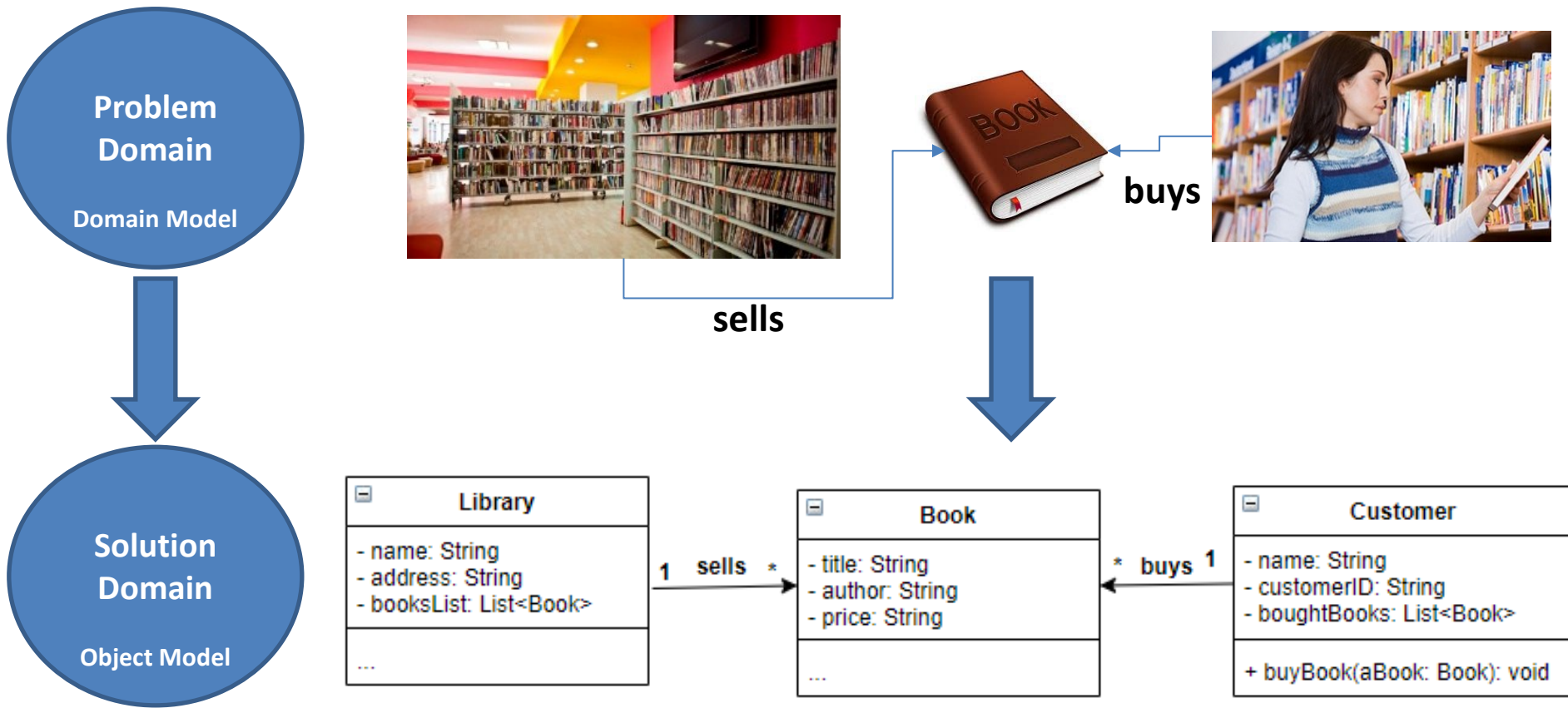


- Problem Domain and Solution Domain



Problem Solving

- Modeling the semantic gap – an object-oriented approach



Adapted from [\[Source\]](#)

Symptoms of Bad Code

- Bad code slows us down
 - Why do we write it?
 - Programmer vs doctor
- Symptoms
 - Confusing – Code review
 - Rigidity – to change something it breaks due to bad dependencies
 - Fragility – breaks in strange places
 - Coupled – difficult to reuse

Code Example

Given a list of integers, return the top 3 distinct largest values in descending order. If fewer than 3 distinct values exist, return all of them.

```
import java.util.*;
public class TopThreeFinder {
    /**
     * Returns the top three largest numbers from a list.
     */
    public static List<Integer> getTopThreeNumbers(List<Integer> numbers) {
        if (numbers == null) {
            return new ArrayList<>();
        }
        // Sort the list in descending order
        List<Integer> sortedNumbers = new ArrayList<>(numbers);
        Collections.sort(sortedNumbers, Collections.reverseOrder());

        List<Integer> result = new ArrayList<>();
        for (Integer num : sortedNumbers) {
            if (!result.contains(num)) {
                result.add(num);
            }
            if (result.size() == 3) {
                break;
            }
        }
        return result;
    }
}
```

UTCN - Programming Techniques

Are there any red flags?

Problems in Code Example

1. Misleading documentation

- Comment says *“top three largest numbers”*
- Requirement is **distinct** values
 - subtle but important mismatch

3. Hidden assumptions

- Assumes sorting is acceptable
- Assumes list size is manageable
- No mention of performance constraints

5. Overconfident polish

- Clean formatting, professional naming
- No tests, no complexity discussion

2. Inefficient algorithm choice

- Full sort $\rightarrow O(n \log n)$
- `result.contains(num)` $\rightarrow$ linear search
- Totally unnecessary if only top 3 values are needed

4. Superficial edge-case handling

- Handles `null`
- But:
 - Empty list?
 - List with one element?
 - Very large input?
 - Duplicates everywhere?

Optimal Single-Pass Java Solution

Idea

Scan once, track the top three distinct values as you go. No sorting. No extra data structures. $O(n)$ time, $O(1)$ space.

Observations

- LLM code is usually reasonable, not reliable
- LLM code optimizes for *plausibility* humans must optimize for *correctness*.
- If you can't explain every line, you don't own the code.

```
public class TopThreeFinder {
    // Returns the top three distinct largest values in descending order.
    public static List<Integer> getTopThreeNumbers(List<Integer> numbers) {
        if (numbers == null || numbers.isEmpty()) {
            return Collections.emptyList();
        }

        Integer first = null; Integer second = null; Integer third = null;
        for (Integer num : numbers) {
            if (num == null) {
                continue; // skip null values safely
            }
            if (num.equals(first) || num.equals(second) || num.equals(third)) {
                continue; // Skip duplicates
            }

            if (first == null || num > first) {
                third = second;
                second = first;
                first = num;
            } else if (second == null || num > second) {
                third = second;
                second = num;
            } else if (third == null || num > third) {
                third = num;
            }
        }

        List<Integer> result = new ArrayList<>();
        if (first != null) result.add(first);
        if (second != null) result.add(second);
        if (third != null) result.add(third);

        return result;
    }
}
```

Optimal Single-Pass Java Solution

Idea

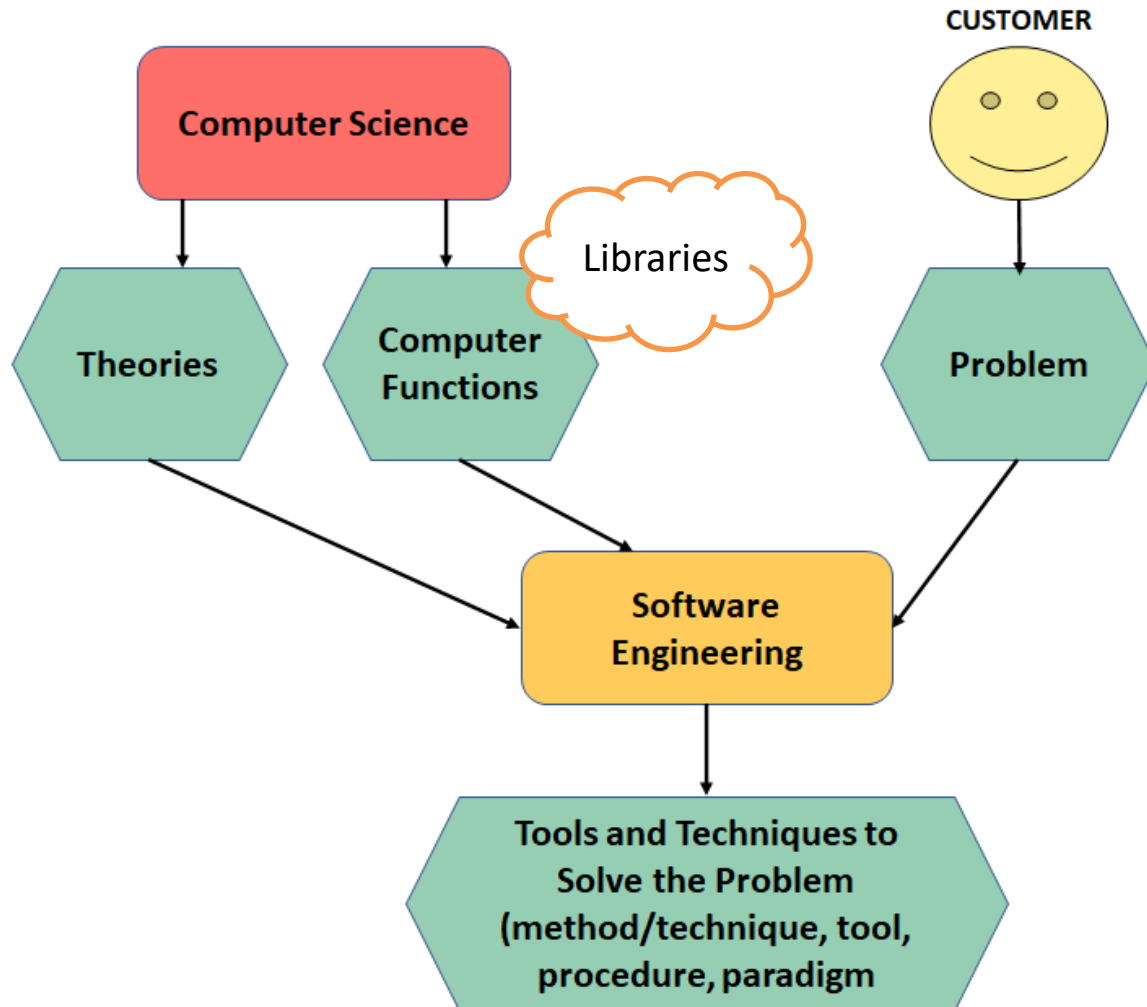
Scan once, track the top three distinct values as you go. No sorting. No extra data structures. $O(n)$ time, $O(1)$ space.

Observations

- LLM code is usually reasonable and reliable
- LLM code optimizes for *probability* humans must care about *correctness*.
- If you can't verify every line, you don't trust the code.

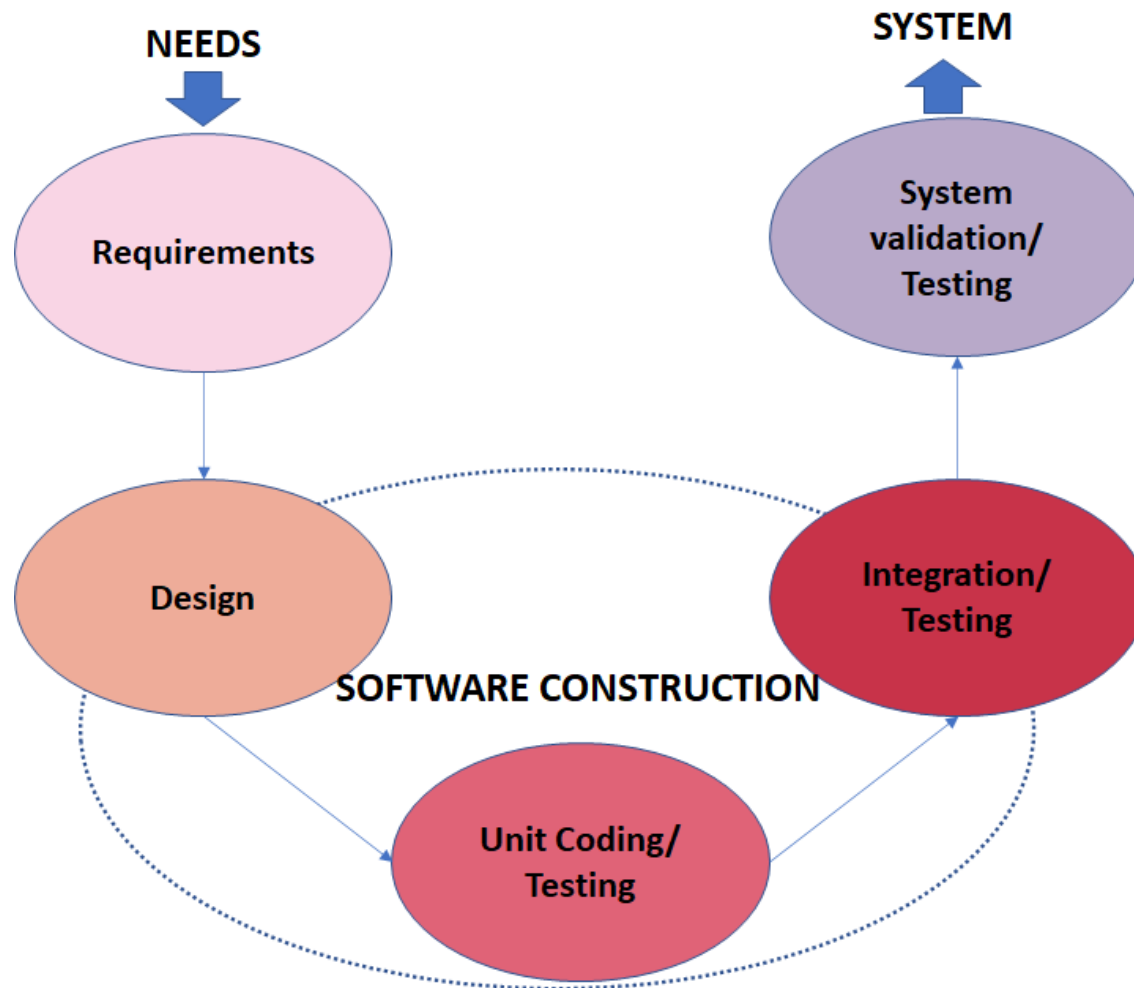
```
public class TopThreeFinder {  
    // Returns the top three distinct largest values in order.  
    public static List<Integer> getTopThreeNumbers(List<Integer> numbers) {  
        if (numbers == null || numbers.isEmpty())  
            return Collections.emptyList();  
        Integer first = null; Integer second = null; Integer third = null;  
  
        for (Integer num : numbers) {  
            if (num == null) continue; // skip null values safely  
  
            // Skip duplicates  
            if (num.equals(first) || num.equals(second) || num.equals(third))  
                continue;  
  
            if (first == null || num > first) {  
                third = second;  
                second = first;  
                first = num;  
            } else if (second == null || num > second) {  
                third = second;  
                second = num;  
            } else if (third == null || num > third) {  
                third = num;  
            }  
        }  
  
        List<Integer> result = new ArrayList<>();  
        if (first != null) result.add(first);  
        if (second != null) result.add(second);  
        if (third != null) result.add(third);  
  
        return result;    }  
}
```

Software Engineering



Adapted from [\[Source\]](#)

Software engineering process



Principles

- Provide value to users
- KISS
- Keep the vision
- Plan ahead and reuse
- Think!

Goal

- High quality

Software engineering process

- Understanding of software development

Building a house analogy

Determine and analyze the residents' requirements

Architect/
engineer

Produce and document the overall design of the house

Produce detailed specifications

Identify and design the components

Build each component of the house

Worker /
programmer

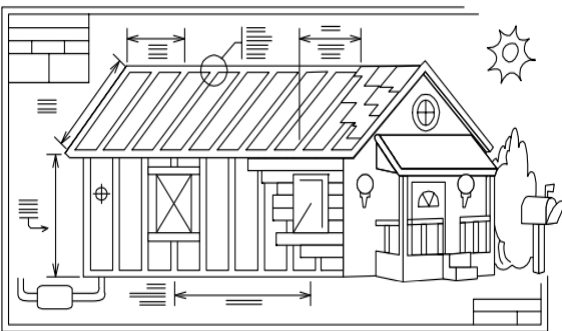
Test each house component

Integrate the components and test as a whole

Make final modifications after the residents have moved in

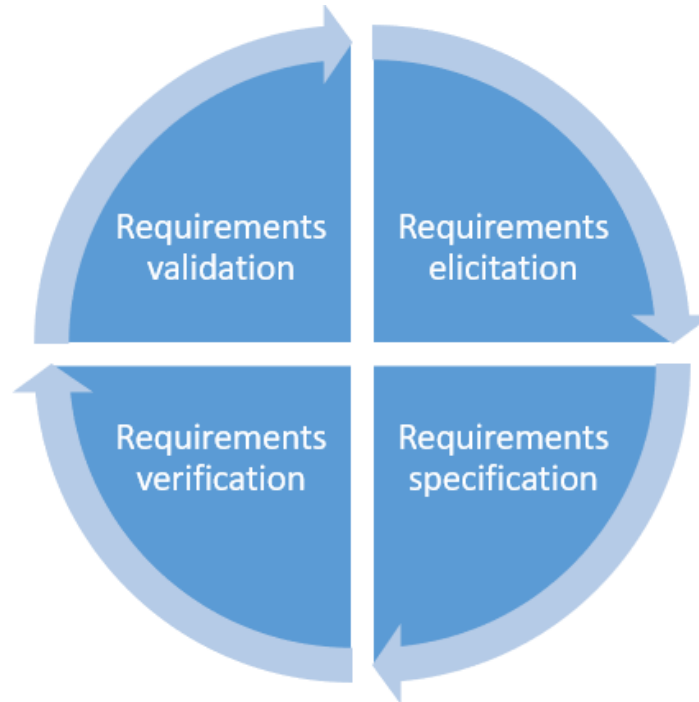
Continue the maintenance by the residents

[\[Source\]](#)



Software engineering process

Requirements Engineering



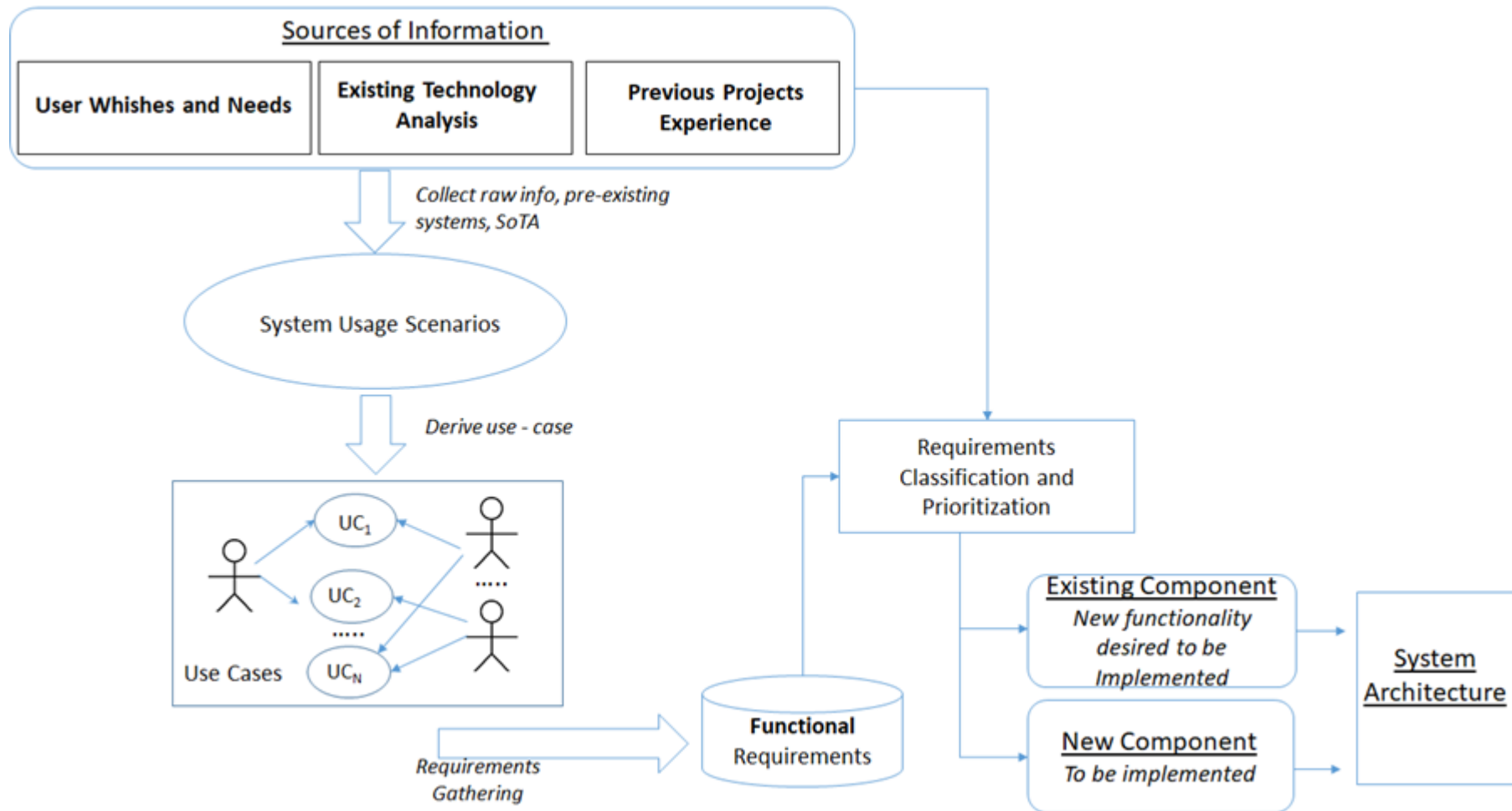
- Types of requirements: functional, non-functional, constraints

Pay attention to requirements to minimize the changes to a system after development begins!

Software engineering process

Requirements Engineering

- Functional requirements and system architecture



Software engineering process

Requirements Engineering

- Requirements Prioritization and Classification – the MoSCoW technique

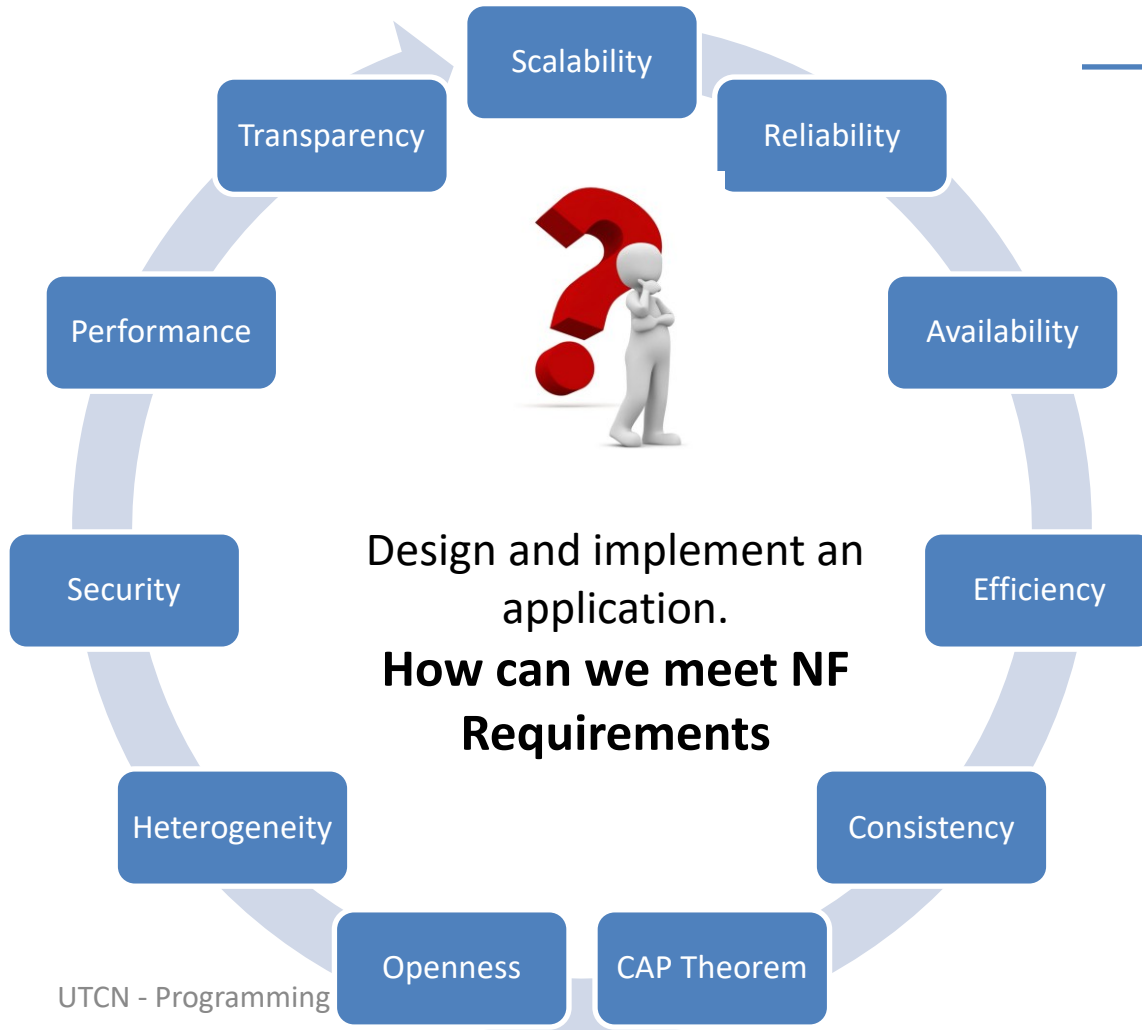


- Trade-offs in software development

Software engineering process

Requirements Engineering

- Nonfunctional requirements



NF in practice

SLI - Service Level Indicators

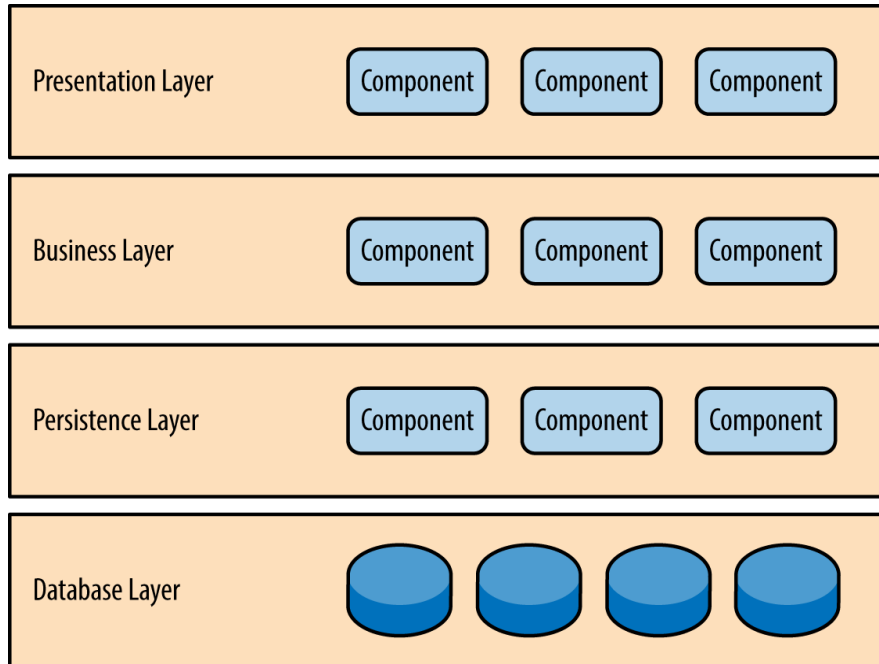
SLO – Service Level Objectives

SLA - Service Level Agreements

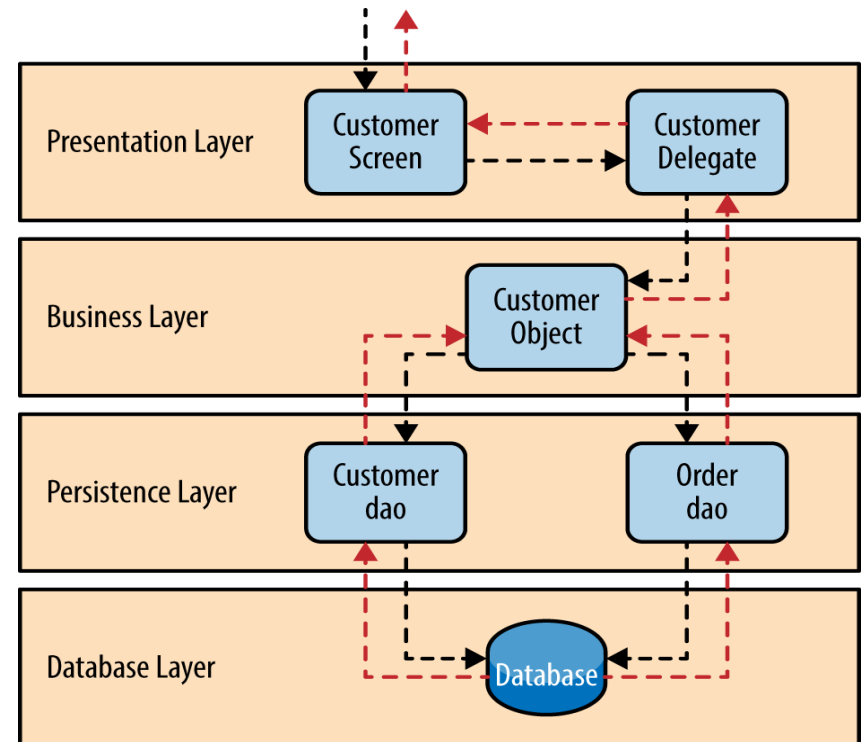
Software engineering process

High level system architecture design

LAYERED ARCHITECTURE



EXAMPLE

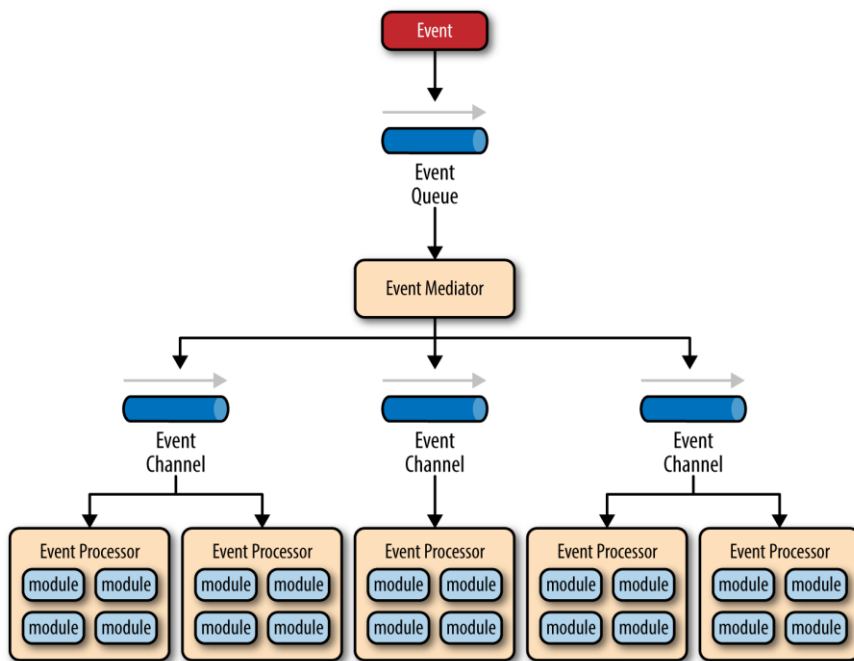


Software engineering process

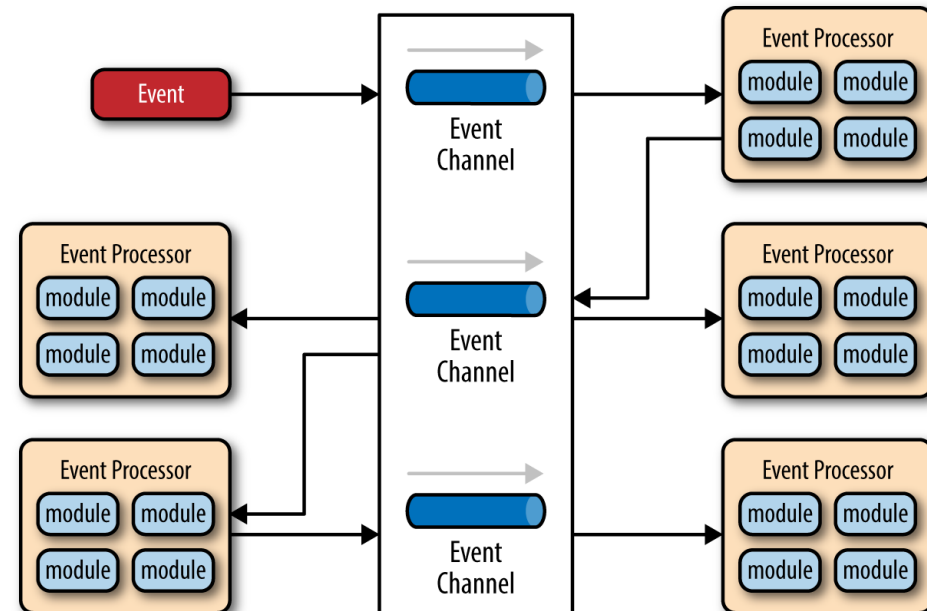
High level system architecture design

- Event driven architecture
 - Asynchronous pattern based on highly decoupled, event processing components

MEDIATOR TOPOLOGY



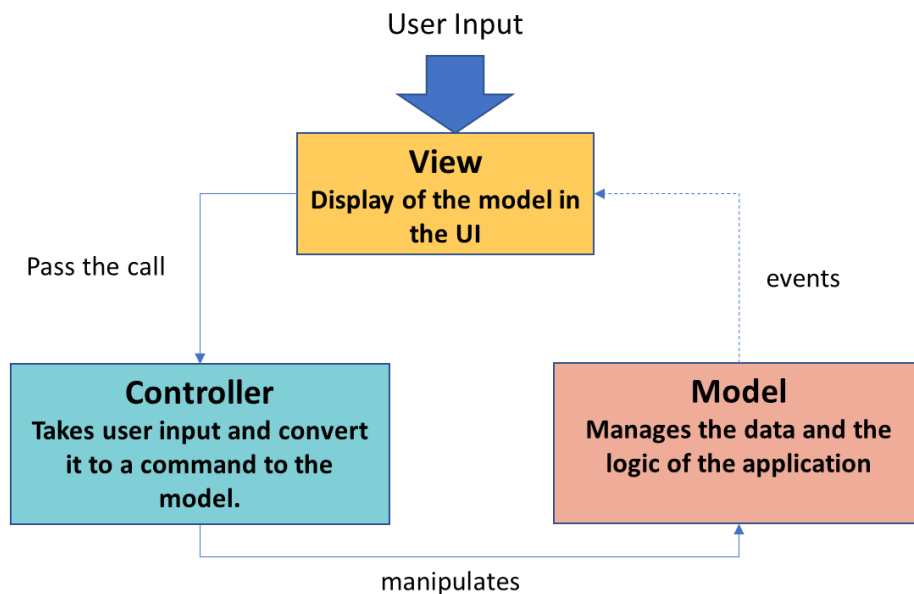
BROKER TOPOLOGY



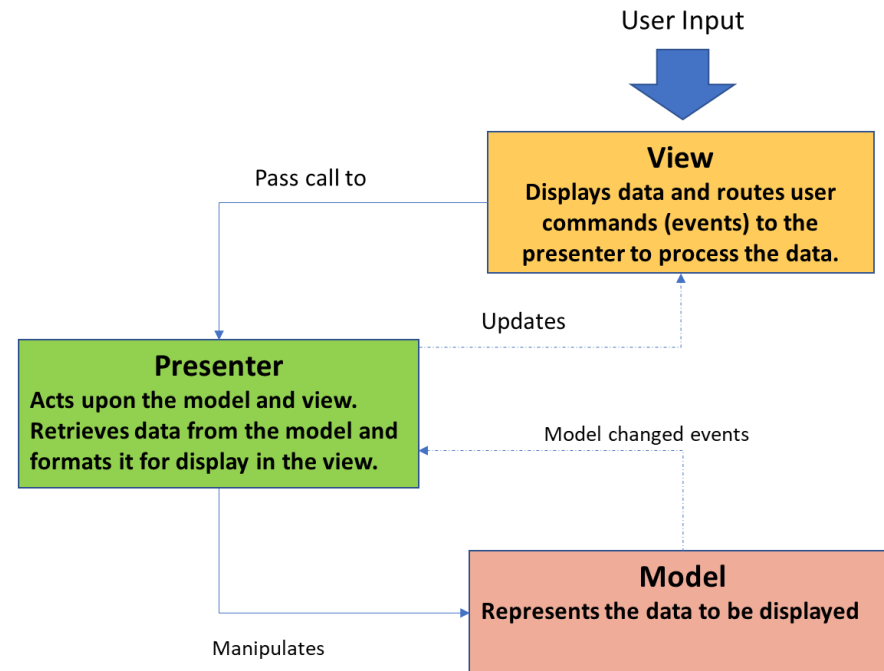
Software engineering process

High level system architecture design

Model View Controller



Model View Presenter



- Efficient for code re-use and parallel development

Software engineering process

Detailed Design

- Objective: Obtain a good (sub)system / component
 - (partial) reuse existent resources;
 - (partial) new resources
- Design
 - iterative try-error and divide and conquer type activity
- Breaking complexity
 - top-down or bottom-up approaches
- Skilled designers
 - Use heuristics
 - Make trade offs

Software engineering process

Detailed Design

1. Software system



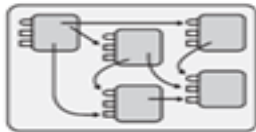
Best practices

2. Division into subsystems/packages



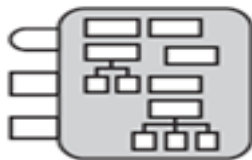
- Minimize the relationships between subsystems;
- Avoid circular relationships
- Consider common design patterns

3. Division into classes within packages



- OOP Paradigms, class level design, design patterns, SOLID principles, etc.
- Loose coupling, high cohesion

4. Division into data and routines within classes



5. Internal routine design



- Code writing rules, clean code

[\[Source\]](#)

Software engineering process

Detailed Design

- Desirable Software Features
 - **External features** – visible to the customer
 - Availability, reliability, performance, fault tolerant, maintainability, security, interoperability, portability, usability, functionality, system integrity, efficiency.
 - **Internal features** – perceptible only to developers
 - minimal complexity, ease of maintenance, extensibility, reusability, high fan-in, low-to-medium fan-out, stratification, standard techniques

External features count in the end, but internal features make it possible to obtain them.

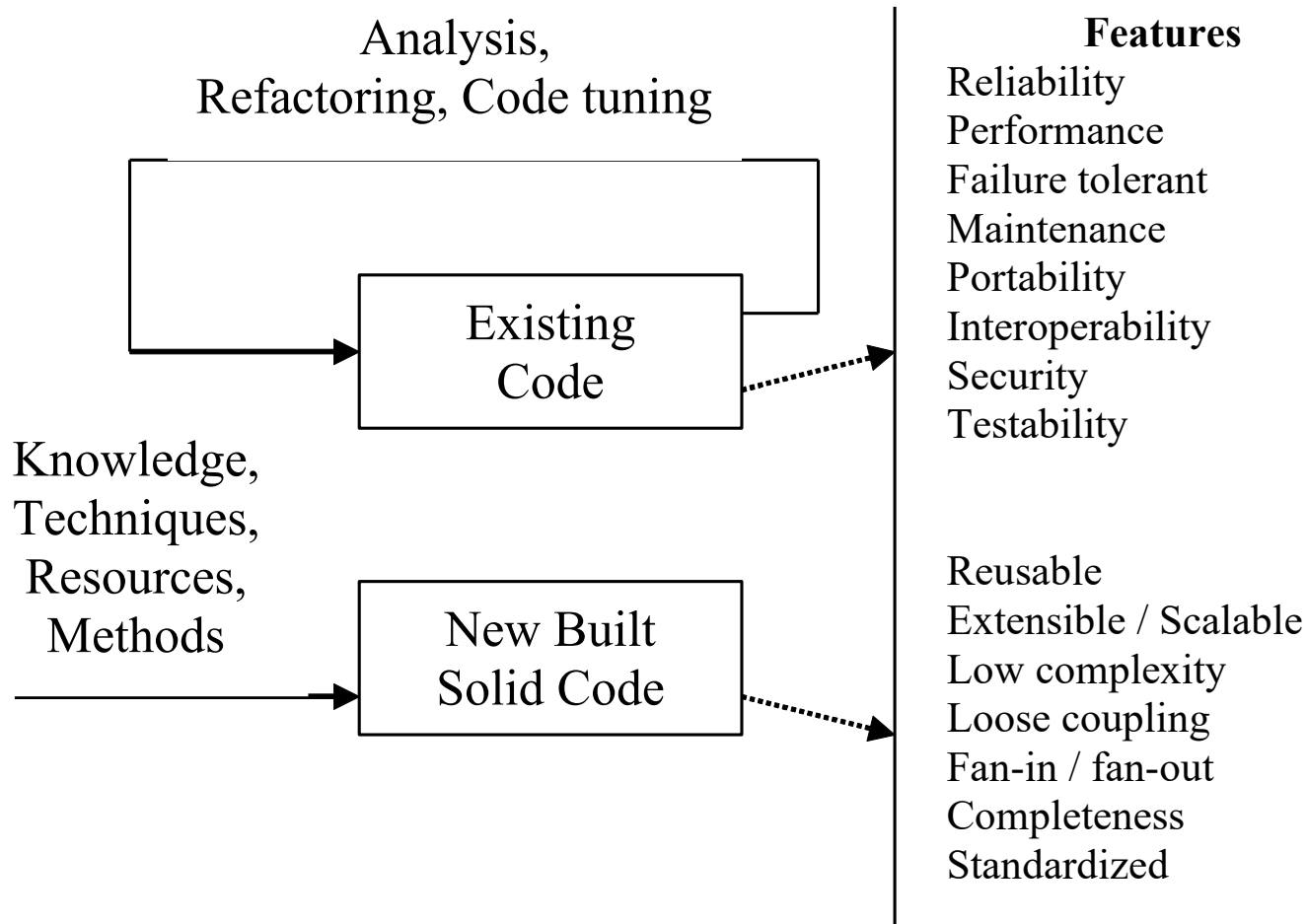
Software engineering process

Detailed Design

- Heuristics
 - Design is nondeterministic => skillful application of an effective set of heuristics is the core activity
 - Find real-world objects
 - Build abstractions and hierarchies of abstractions
 - Encapsulate implementation details
 - Inherit or delegate
 - Keep coupling loose and coherence high
 - Look for common design patterns
 - Think of associating things
 - Design for test
 - Draw diagrams
 - Keep the design modular

Software engineering process

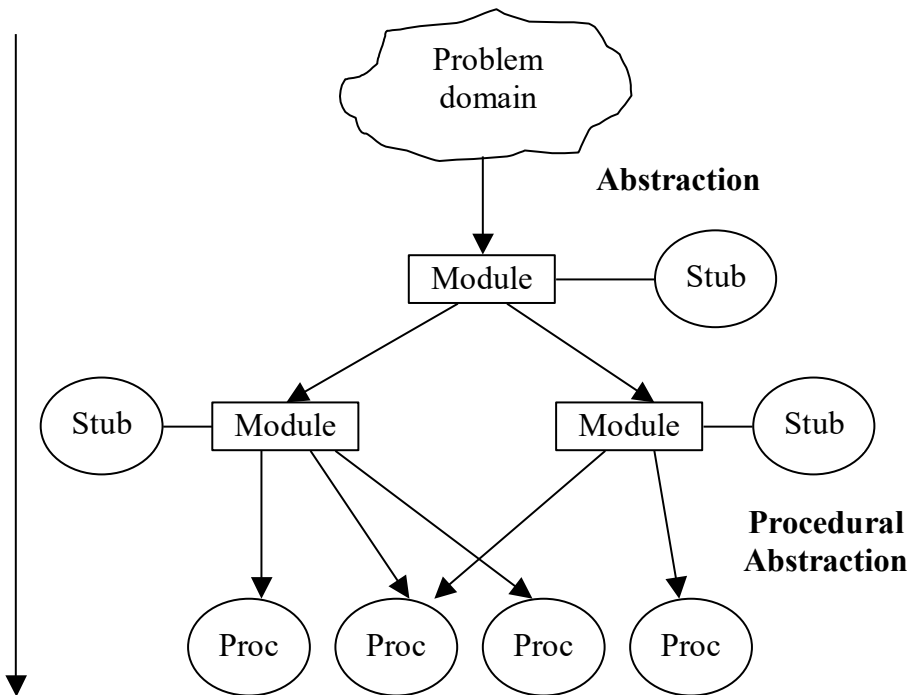
Implementation



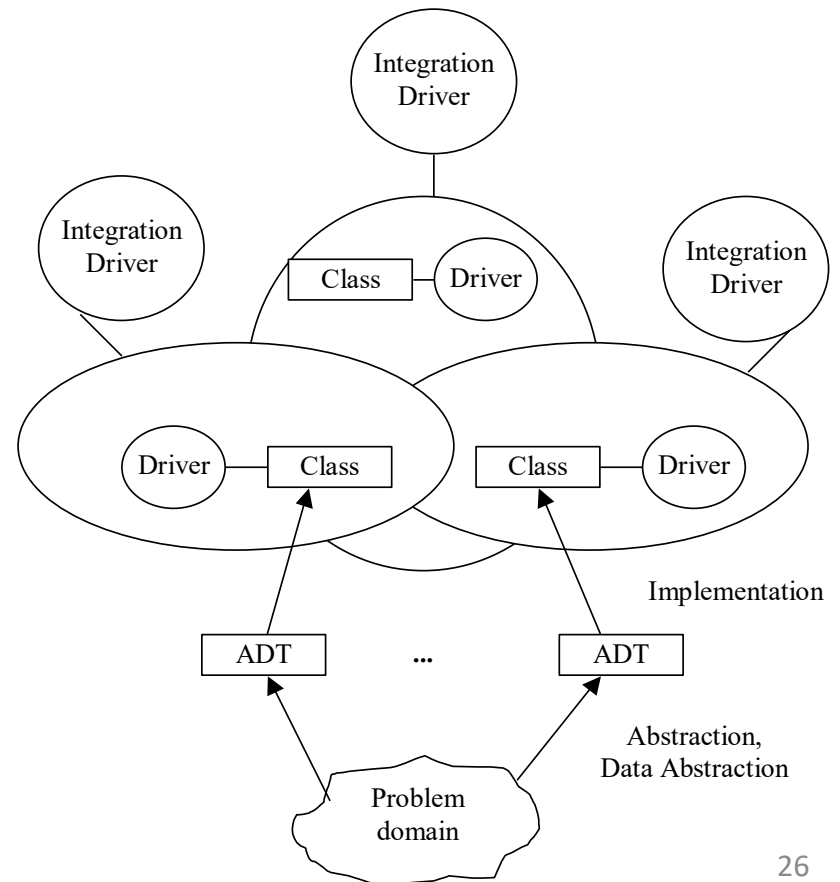
Software engineering process

Implementation

Divide et Impera - Top Down
(Procedural Development)



Divide et Impera – Bottom Up
(OO development))



Software engineering process

The Pragmatic Software Engineer

The Pragmatic Programmer

From Journeyman to Master

Andrew Hunt
David Thomas

Characteristics

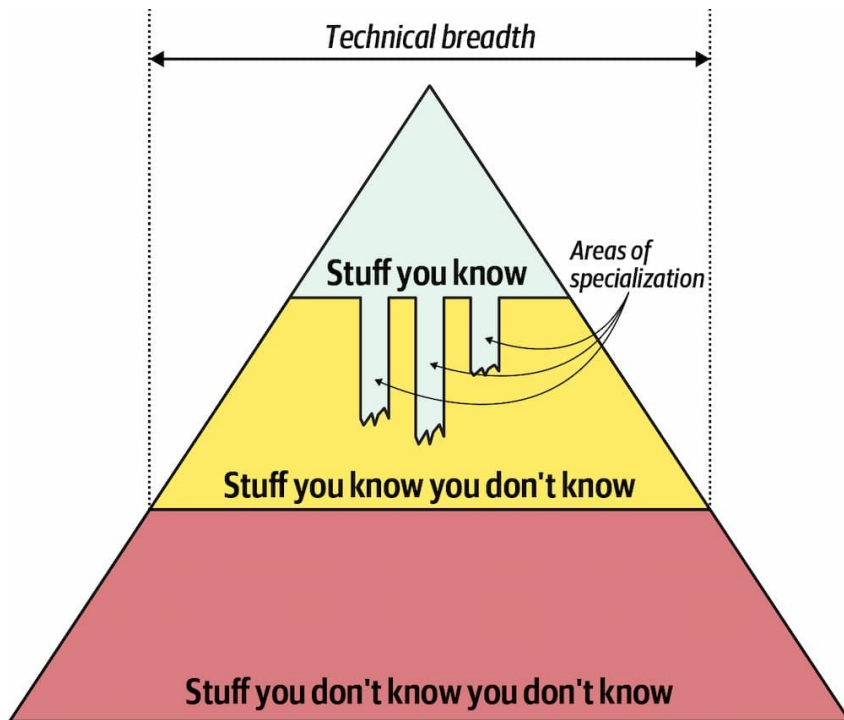
- Early adopter/fast adapter
- Tend to ask questions
- Critical thinker
- Realistic
- Try hard to be familiar with a broad range of technologies

TIP1: Care about your CODE

TIP2: Think! About Your Work

Software engineering process

The Pragmatic Programmer

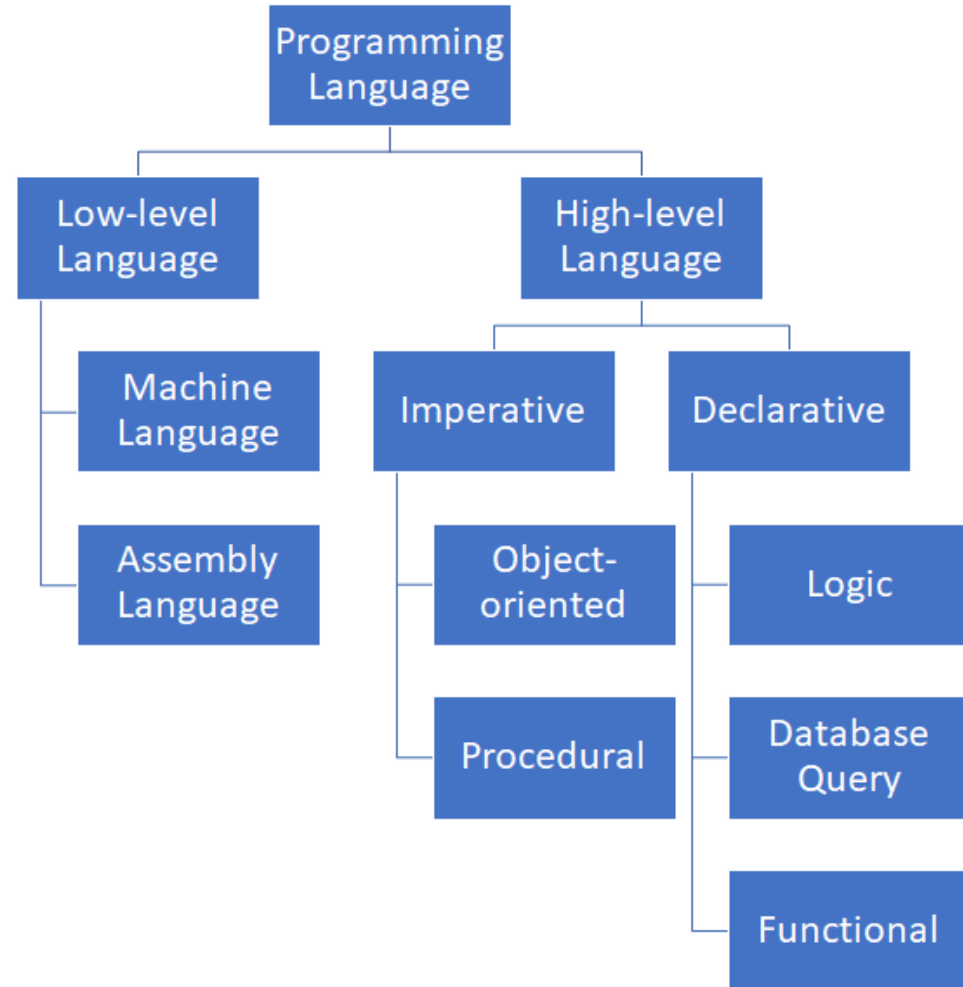


Fundamentals of Software Architecture: An Engineering Approach by Mark Richards

Software engineering process

Implementation – Programming Languages

- **Choosing the right programming language**
 - Difficult task
 - Decision metrics
 - Ease of learning,
 - ease of understanding,
 - speed of development,
 - portability,
 - fit-for-purpose



Many languages are multi-paradigm!

Software engineering process

Implementation – Programming Languages

Imperative Programming

- Uses statements that change a program's state
- Every step of the program is executed sequentially
- Sub-categories
 - Procedural: relies on procedures, algorithms - ex. Pascal, C++
 - Object-oriented: relies on classes, objects containing data and methods (procedures) – ex. C++, Java, C#
- Java is imperative and object oriented

```
List<Integer> input = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> output = new ArrayList<>();
for (Integer x : input) {
    if (x % 2 == 0) {
        output.add(x);
    }
}
```

Software engineering process

Implementation – Programming Languages

Declarative Programming

- The programmer declares/describes the properties of the result but not actually how to compute the result
- Sub-categories
 - Functional: relies on functions, Lambda Calculus – e.g., Lisp, ML, Haskell
 - Logic: uses predicate calculus – e.g., Prolog
 - Database Query: e.g., SQL
- Imperative paradigm versus declarative paradigms

Imperative

```
var numbers = [1,2,3,4,5]
var doubled = []

for(var i = 0; i < numbers.length; i++) {
  var newNumber = numbers[i] * 2
  doubled.push(newNumber)
}
console.log(doubled) //=> [2,4,6,8,10]
```

Declarative

```
var numbers = [1,2,3,4,5]

var doubled = numbers.map(function(n) {
  return n * 2
})
console.log(doubled) //=> [2,4,6,8,10]
```

[\[Source\]](#)

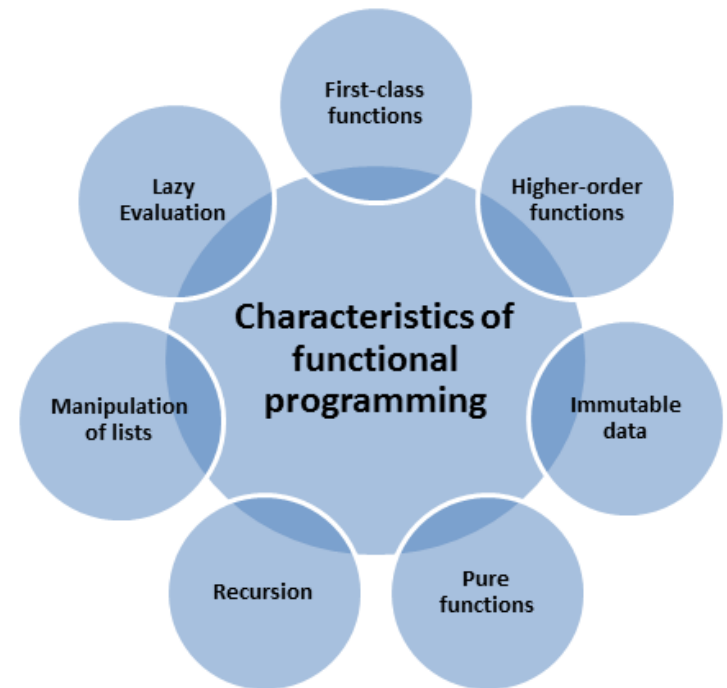
Software engineering process

Implementation – Programming Languages

Declarative Functional Programming

```
List<Integer> input = Arrays.asList(1, 2, 3, 4, 5);  
var output = input.where( x -> x % 2 == 0);
```

- No sequence of steps
- A function ($x \% 2 == 0$) is passed as a parameter to another function (`where()`) that is applied to an object (`input`)
- The evaluation does not change the internal program state
- Pure functional languages: Haskell, Hope, Mercury
- Java is not pure functional language

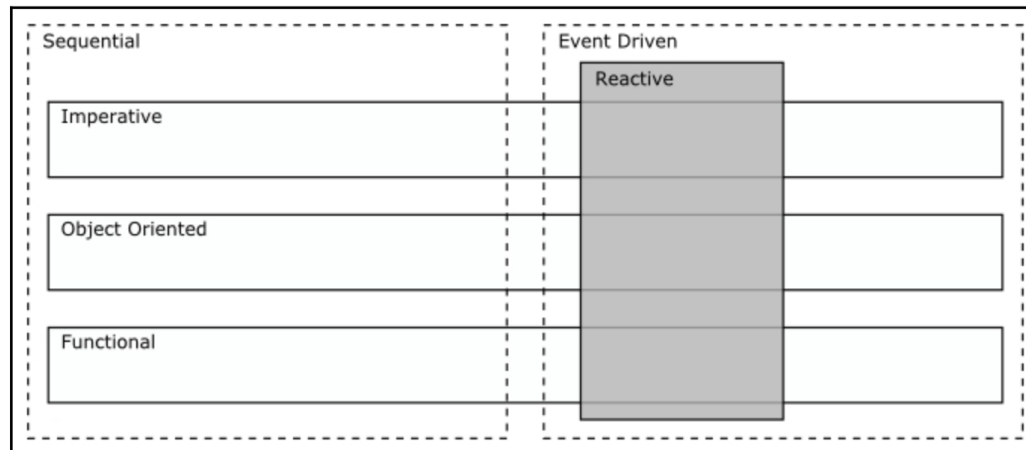


Software engineering process

Implementation – Programming Languages

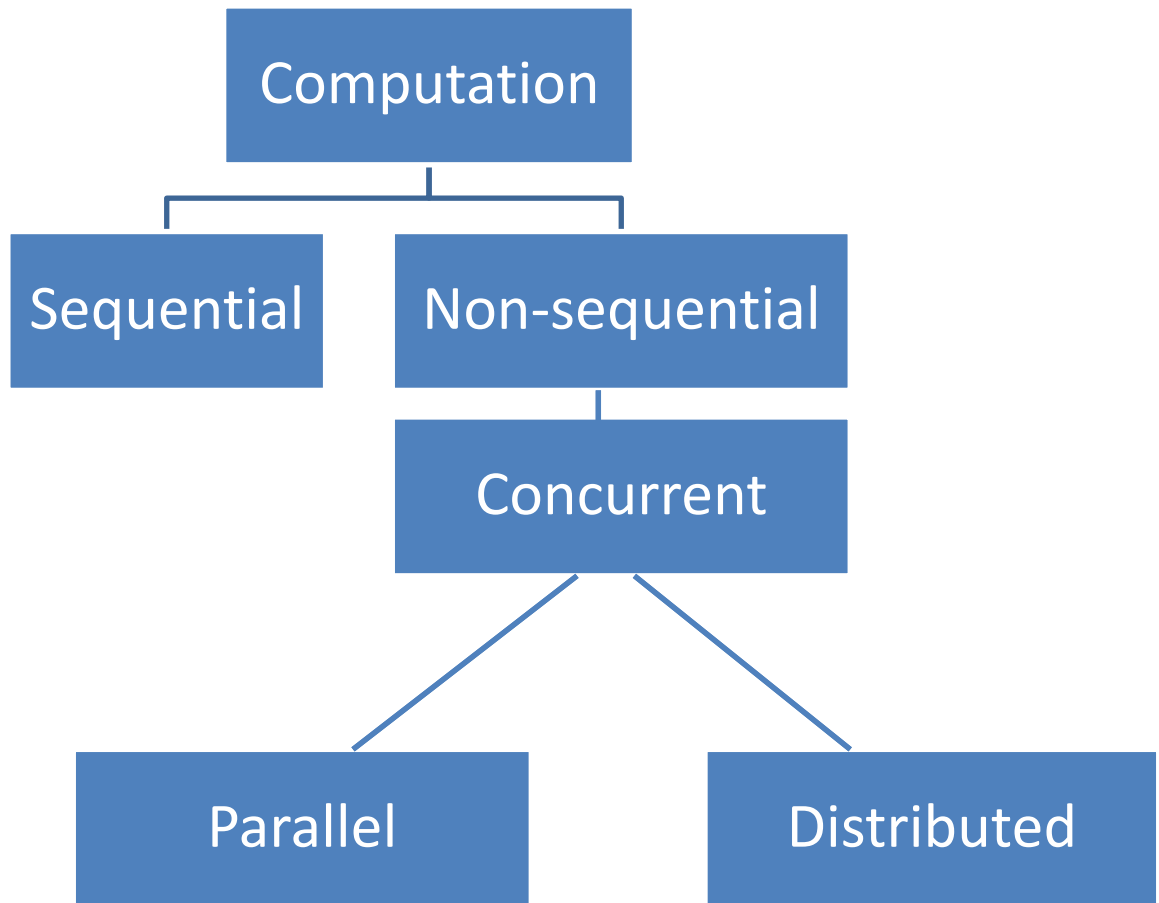
Reactive Programming

- Deals with data flows and propagation of data changes (data streams)
- Consider the assignment statement: $x = y + z;$
 - If y and z changes values $\Rightarrow$ x value will be modified accordingly
- Principles
 - Code “reacts” to events
 - Handles values as they vary in time, propagating changes to the code that uses those values
- Reactive programming – is not a programming paradigm of its own



Software engineering process

Implementation – Types of Computation

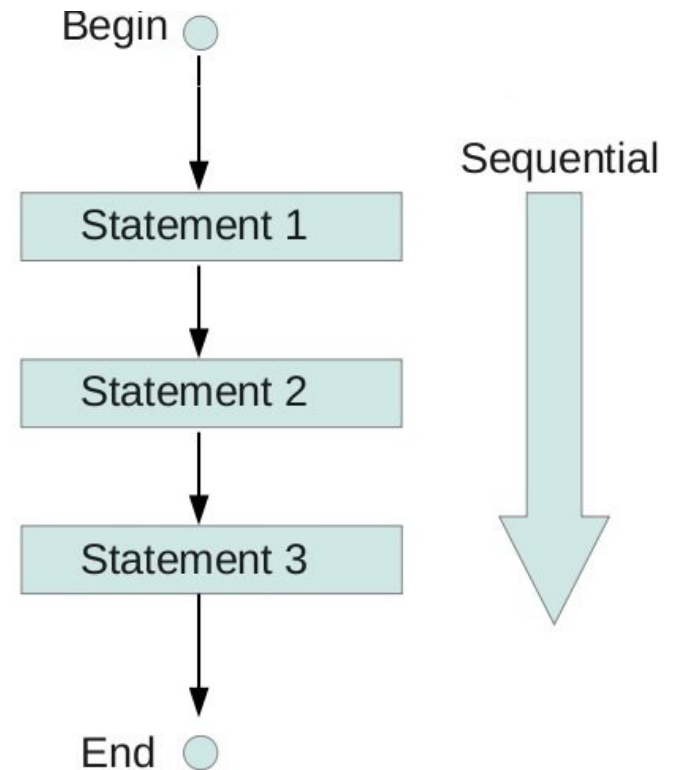


Software engineering process

Implementation – Types of Computation

- **Sequential Programming**

- Execution of one statement at a time
- Usually executed on a single processor
 - Processor speed is important
- Transforms input data to results according to the implemented algorithm
- Usually, deterministic



Software engineering process

Implementation – Types of Computation

- **Concurrent Programming**

- Concurrent program
 - Tasks executed simultaneously (in parallel)
- Execution support
 - **Parallel** or **distributed** processes
- Transform sequential to concurrent programming
 - Decompose large tasks into multiple smaller tasks
 - Assigning the smaller tasks to multiple workers to work on simultaneously
 - Mapping on computational resources
 - Coordinating the workers

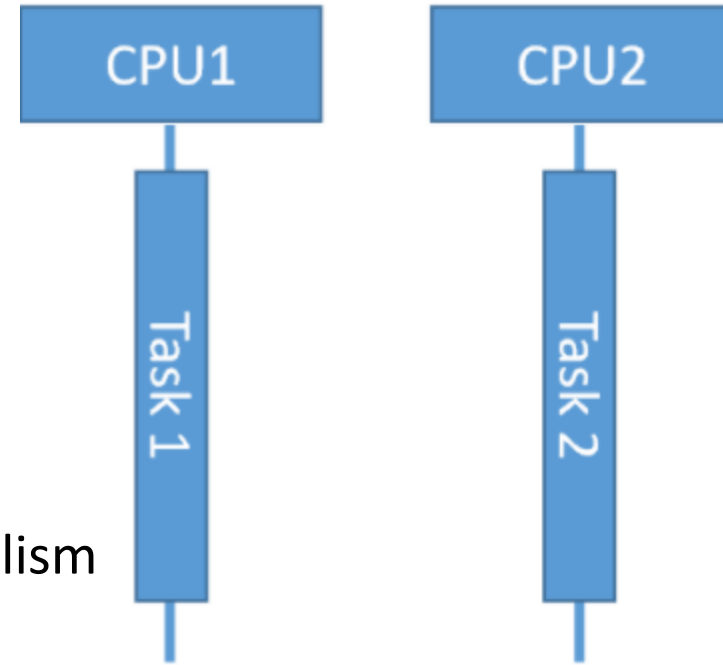


Software engineering process

Implementation – Types of Computation

- **Parallel Programming**

- Special form of concurrent programming
 - Multiple physical processors
- Constructing parallel applications
 - Functional parallelism
 - Master–slave parallelism
 - Single Program Multiple Data parallelism
- Parallel programming main tasks
 - Decomposing tasks
 - Distributing tasks
 - Coordinating tasks

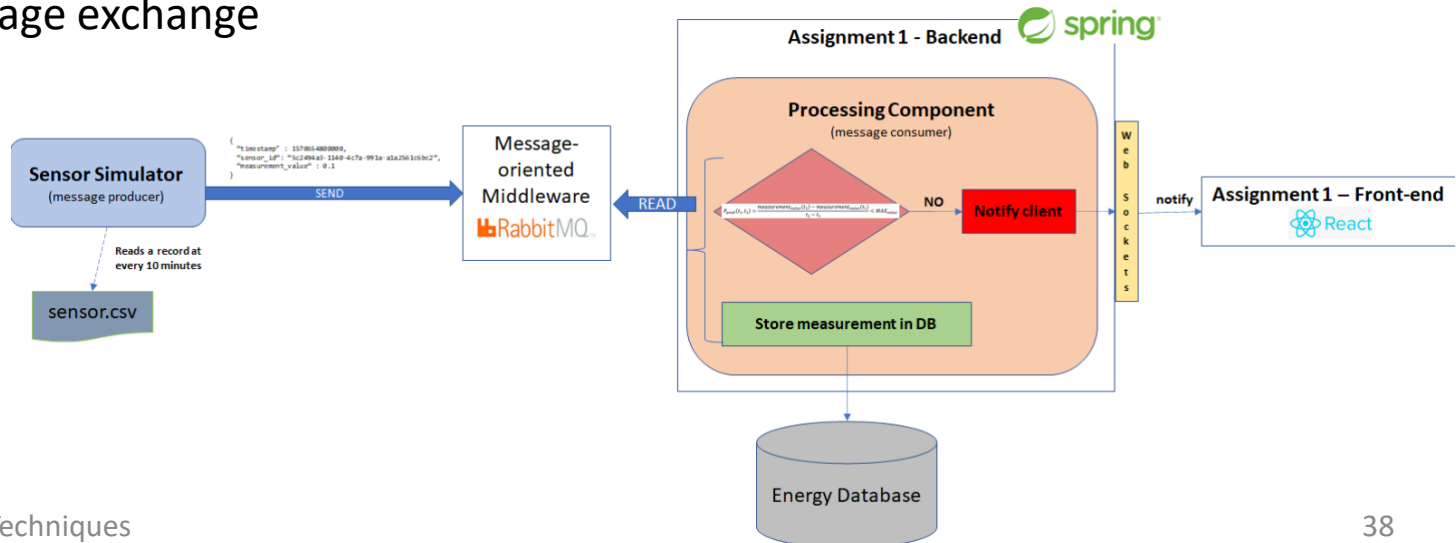
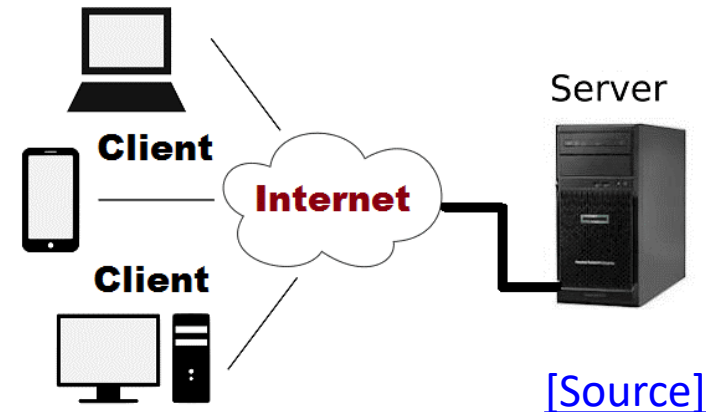


Software engineering process

Implementation – Types of Computation

- **Distributed Programming**

- Special form of concurrent programming
 - multiple physical processors
 - remotely located
 - no shared memory
- Inter-process communication
 - communication channels
 - message exchange

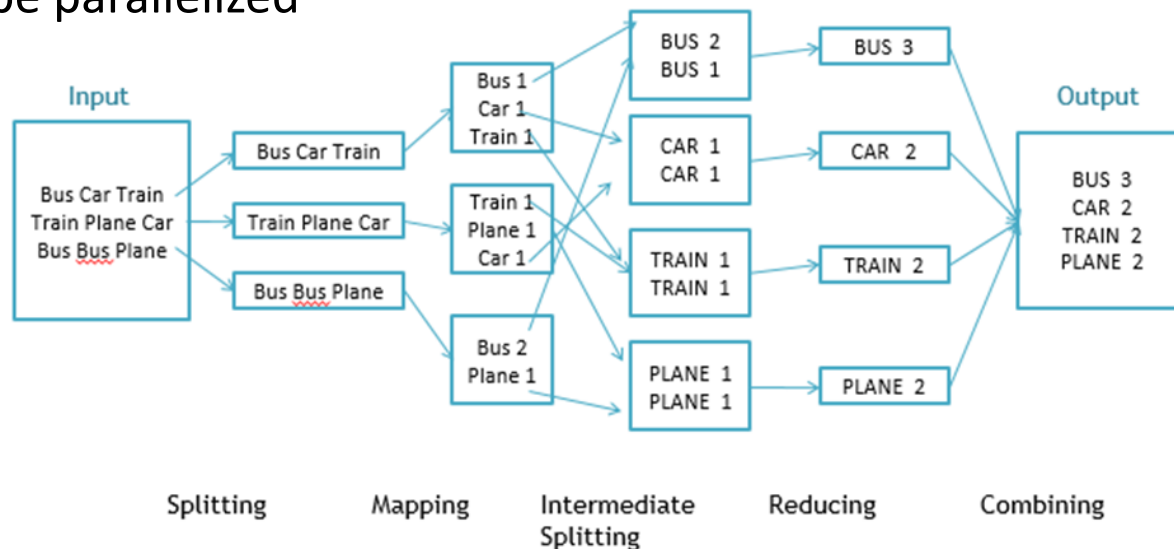


Software engineering process

Implementation – Types of Computation

- **Transformational (pipeline) Programming**

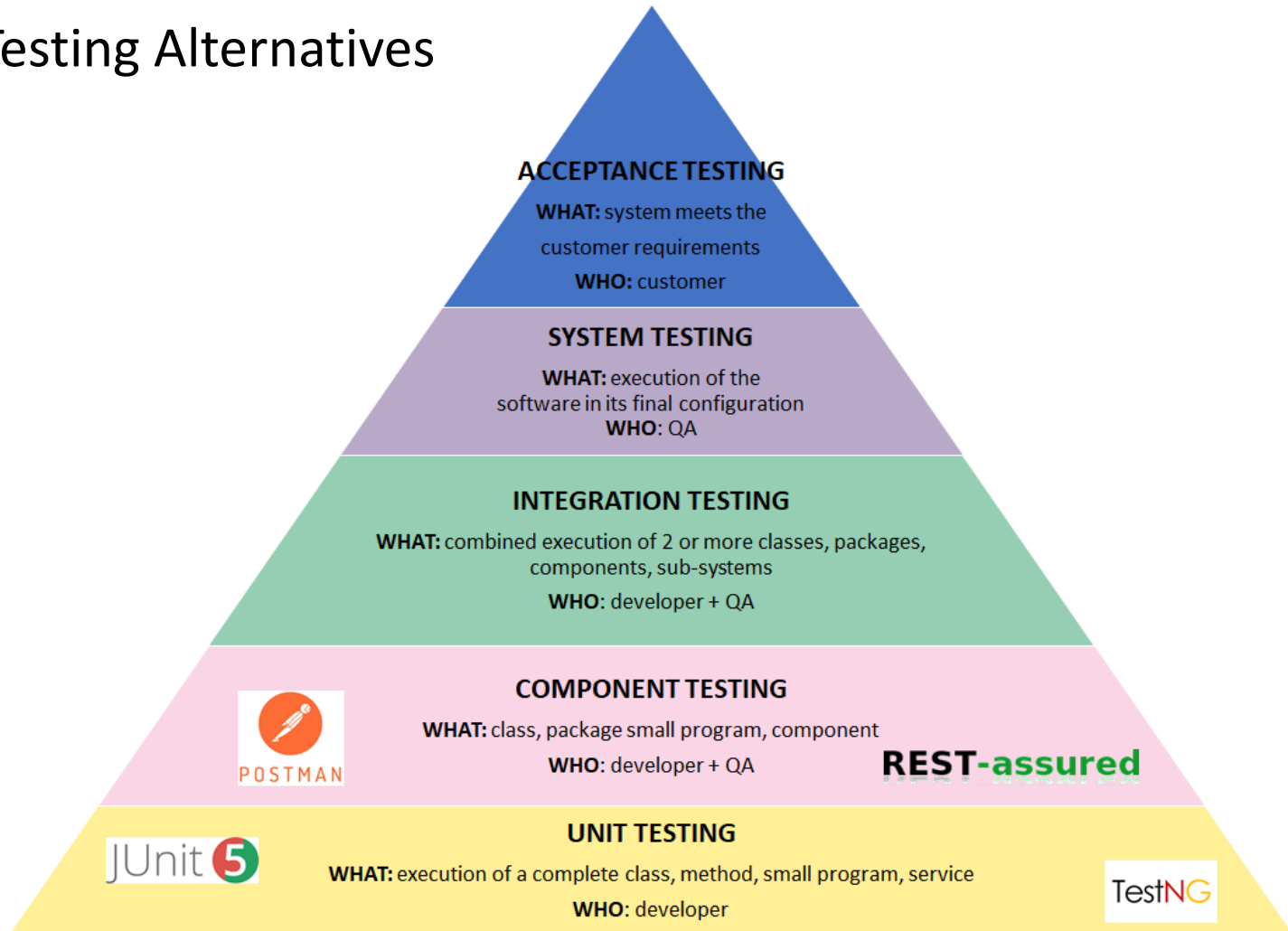
- Data driven
- Based on the sequence: read data -> process data -> output results
- Can be seen as functions of n inputs generating m outputs
- Program behavior depends on program current state and input
- Can be parallelized



Software engineering process

Testing and Debugging

- Testing Alternatives



Software engineering process

Testing and Debugging

Debugging



- Type of errors
- Techniques
 - Pay attention to the error message
 - Use debuggers
 - Add debug messages or comment part of code
 - Check previous working versions
 - Rubber duck
 - Reproduce the problem
 - **Take a break**

[\[Source\]](#)